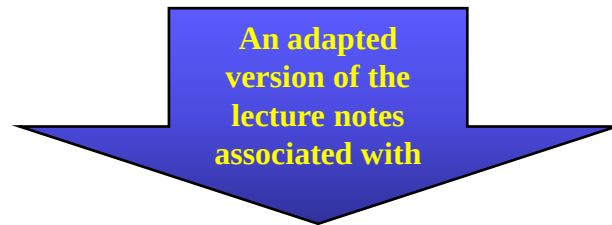




Selections



The **boolean** Type and Operators

- Often in a program you need to compare two values, such as `i > j`.
- Java provides six comparison operators (also known as relational operators) that can be used to compare two values.
- The result of the comparison is a boolean value: true or false.

```
boolean b = (1 > 2) ;
```



Relational Operators

Java Operator	Mathematics Symbol	Name	Example (radius is 5)	Result
<	<	less than	<code>radius < 0</code>	<code>false</code>
<=	=	less than or equal to	<code>radius <= 0</code>	<code>false</code>
>	>	greater than	<code>radius > 0</code>	<code>true</code>
>=	=	greater than or equal to	<code>radius >= 0</code>	<code>true</code>
==	=	equal to	<code>radius == 0</code>	<code>false</code>
!=	?	not equal to	<code>radius != 0</code>	<code>true</code>



Problem: A Simple Math Learning Tool

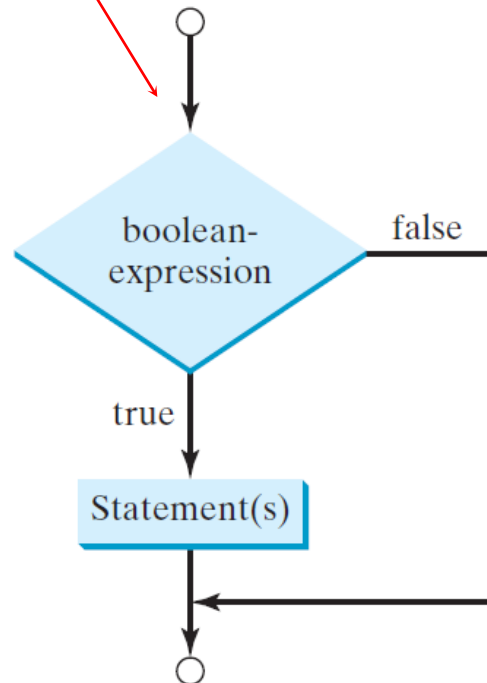
This example creates a program to let a first grader practice additions. The program randomly generates two single-digit integers number1 and number2 and displays a question such as “What is 7 + 9?” to the student. After the student types the answer, the program displays a message to indicate whether the answer is true or false.

```
import java.util.Scanner;
public class AdditionQuiz {
    public static void main(String[] args) {
        int number1 = (int)(System.currentTimeMillis() % 10);
        int number2 = (int)(System.currentTimeMillis() / 7 % 10);
        Scanner input = new Scanner(System.in);
        System.out.print( "What is " + number1 + " + " + number2 + "? ");
        int answer = input.nextInt();
        Boolean result = (number1 + number2 == answer);
        System.out.println(
            number1 + " + " + number2 + " = " + answer + " is " + result); } }
```

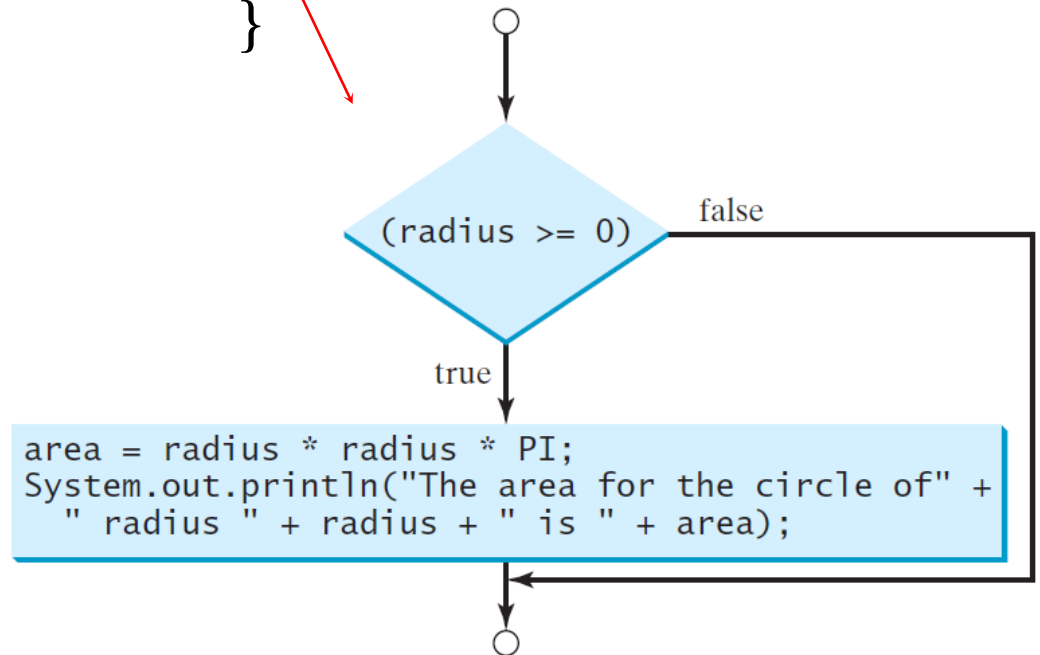


One-way **if** Statements

```
if (boolean-expression) {  
    statement(s);  
}
```



```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area"  
        + " for the circle of radius "  
        + radius + " is " + area);  
}
```



Note

```
if i > 0 {  
    System.out.println("i is positive");  
}
```

(a) Wrong

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(b) Correct

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(a)

Equivalent

```
if (i > 0)  
    System.out.println("i is positive");
```

(b)



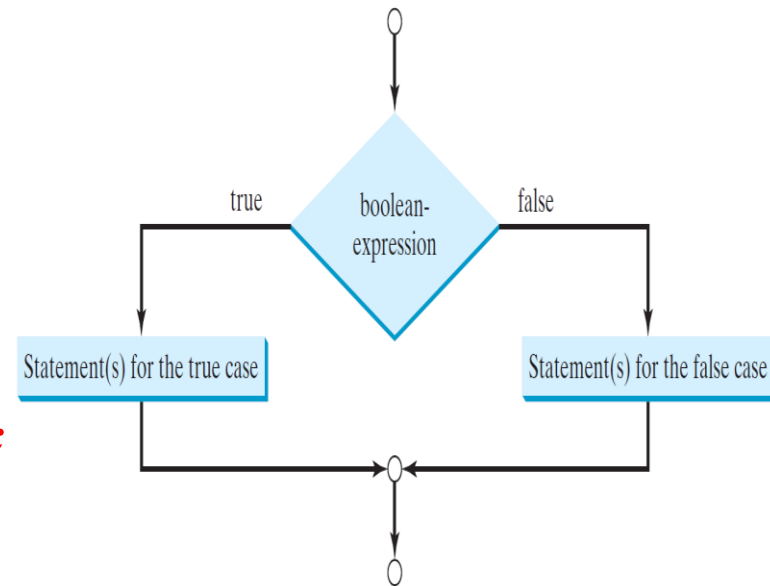
Simple if Demo

Write a program that prompts the user to enter an integer. If the number is a multiple of 5, print HiFive. If the number is divisible by 2, print HiEven.



The Two-way if

```
if (boolean-expression) {  
    statement(s)-for-the-true-case;  
}  
else {  
    statement(s)-for-the-false-case;  
}
```



```
if (radius >= 0) {  
    area = radius * radius * 3.14159;
```

```
    System.out.println("The area for the "  
        + "circle of radius " + radius +  
        " is " + area);  
}  
else {  
    System.out.println("Negative input");  
}
```

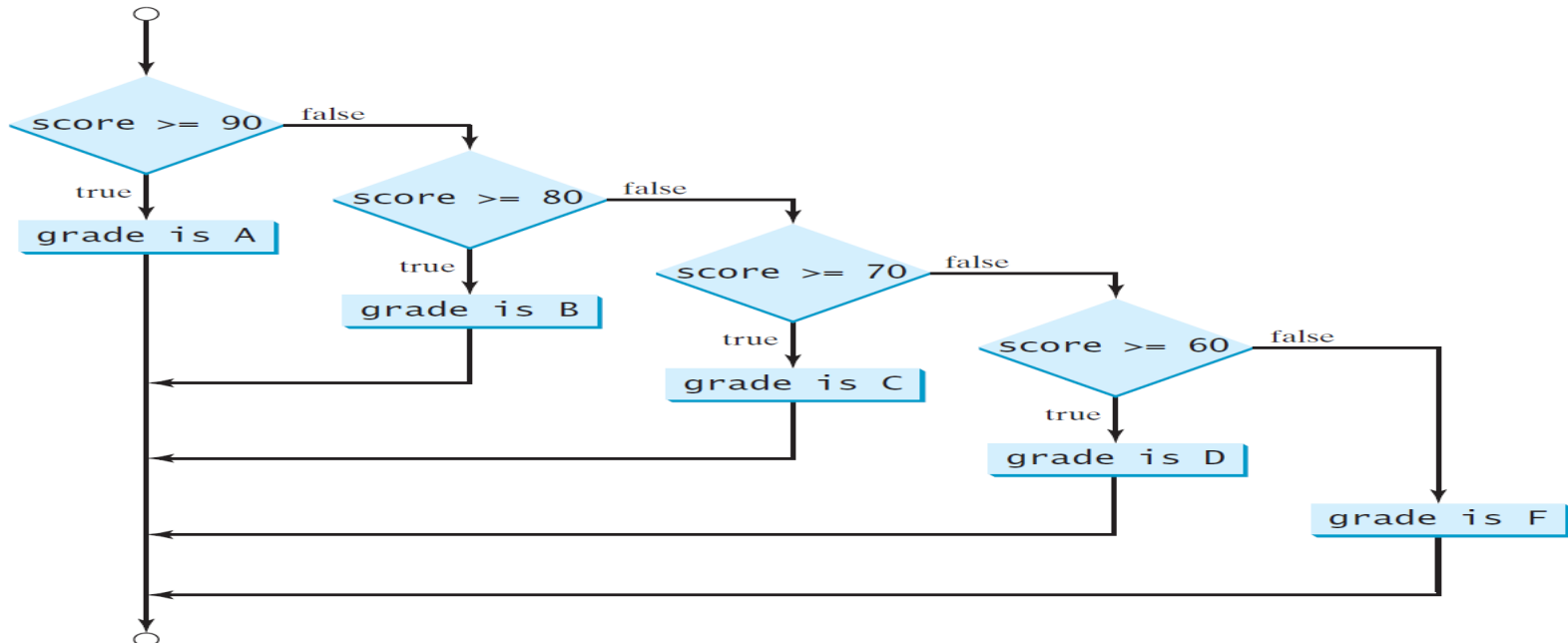

Multiple Alternative if Statements

```
if (score >= 90.0)
    System.out.print("A");
else{
    if (score >= 80.0)
        System.out.print("B");
    else
        if (score >= 70.0)
            System.out.print("C");
        else
            if (score >= 60.0)
                System.out.print("D");
            else
                System.out.print("F");
}
```

Equivalent

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```

This is better



Trace if-else statement

Suppose score is 70.0

The condition is false

```
if (score >= 90.0)
```

```
    System.out.print("A");
```

```
else if (score >= 80.0)
```

```
    System.out.print("B");
```

```
else if (score >= 70.0)
```

```
    System.out.print("C");
```

```
else if (score >= 60.0)
```

```
    System.out.print("D");
```

```
else
```

```
    System.out.print("F");
```



Trace if-else statement

Suppose score is 70.0

The condition is false

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```



Trace if-else statement

Suppose score is 70.0

The condition is true

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```



Trace if-else statement

Suppose score is 70.0

grade is C

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```



Trace if-else statement

Suppose score is 70.0

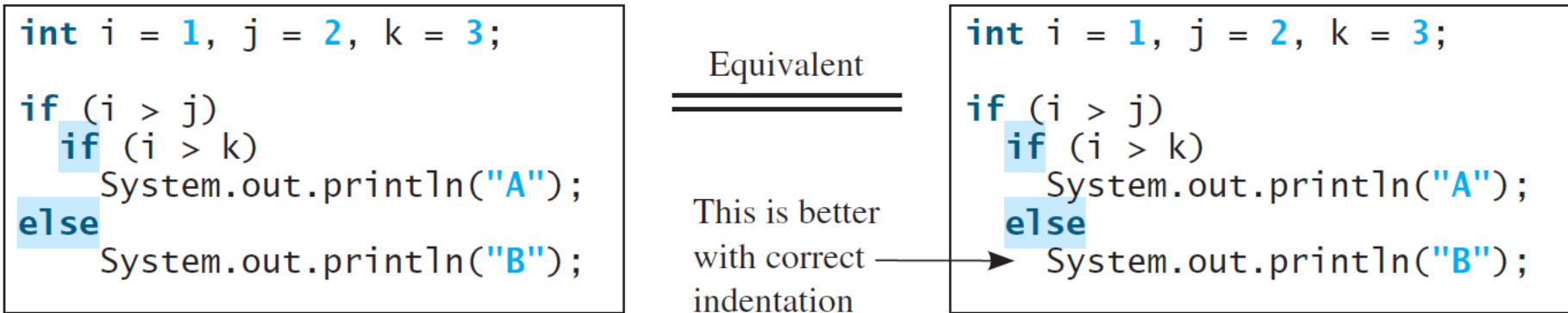
Exit the if statement

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```



Note

The else clause matches the most recent if clause in the same block.



Nothing is printed from the preceding statement. To force the else clause to match the first if clause, you must add a pair of braces:

```
int i = 1, j = 2, k = 3;
if (i > j) { if (i > k)
            System.out.println("A");
        }
else
    System.out.println("B");
```

This statement prints B.



Common Errors

Adding a semicolon at the end of an if clause is a common mistake.

```
if (radius >= 0); ← Wrong
{
    area = radius*radius*PI;
    System.out.println(
        "The area for the circle of radius " +
        radius + " is " + area);
}
```

This mistake is hard to find, because it is not a compilation error or a runtime error, it is a **logic error**.

This error often occurs when you use the next-line block style.

TIP

```
if (number % 2 == 0)
    even = true;
else
    even = false;
```

(a)

Equivalent

```
boolean even
    = number % 2 == 0;
```

(b)

CAUTION

```
if (even == true)
    System.out.println(
        "It is even.");
```

(a)

Equivalent

```
if (even)
    System.out.println(
        "It is even.");
```

(b)

Logical Operators

Operator	Name	Description
!	not	logical negation
&&	and	logical conjunction
	or	logical disjunction
^	exclusive or	logical exclusion

Truth Table for

Operator NOT (!)

p	!p
false	true
true	false

Operator AND (&&)

p₁	p₂	p₁ && p₂
false	false	false
false	true	false
true	false	false
true	true	true

Operator OR (||)

p1	p2	p1 p2
false	false	false
false	true	true
true	false	true
true	true	true

Operator XOR (^)

p1	p2	p1 ^ p2
false	false	false
false	true	true
true	false	true
true	true	false



Examples

Here is a program that checks whether a number is divisible by 2 and 3, whether a number is divisible by 2 or 3, and whether a number is divisible by 2 or 3 but not both:

```
if ((number % 2 == 0) && (number % 3 == 0))  
    System.out.println(number + " is divisible by 2 and 3");  
else if ((number % 2 == 0) || (number % 3 == 0))  
    System.out.println(number + " is divisible by 2 or 3");  
if ((number % 2 == 0) ^ (number % 3 == 0))  
    System.out.println(number + " is divisible by 2 or 3, but not both");
```

Problem: Determining Leap Year?

This program first prompts the user to enter a year as an int value and checks if it is a leap year.

A year is a leap year if it **is divisible by 4** but **not by 100**, or it is divisible by **400**.

$(\text{year} \% 4 == 0 \ \&\& \ \text{year} \% 100 \neq 0) \ ||$

$(\text{year} \% 400 == 0)$



The **switch** Statement

- The **switch** statement is the only other kind of Java statement that implements *multiway* branching
 - When a **switch** statement is evaluated, one of a number of different branches is executed
 - The choice of which branch to execute is determined by a *controlling expression* enclosed in parentheses after the keyword **switch**
 - The controlling expression must evaluate to a **char**, **int**, **short**, or **byte**



The **switch** Statement

- Each branch statement in a **switch** statement starts with the reserved word **case**, followed by a *constant* called a *case label*, followed by a colon, and then a sequence of statements
 - Each case label must be of the same type as the controlling expression
 - Case labels need not be listed in order or span a complete interval, but each one may appear only once
 - Each sequence of statements may be followed by a **break** statement (**break** ;)



The **switch** Statement

- There can also be a section labeled **default**:
 - The **default** section is optional, and is usually last
 - Even if the case labels cover all possible outcomes in a given **switch** statement, it is still a good practice to include a **default** section
 - It can be used to output an error message, for example
- When the controlling expression is evaluated, the code for the case label whose value matches the controlling expression is executed
 - If no case label matches, then the only statements executed are those following the **default** label (if there is one)



The **switch** Statement

- The **switch** statement ends when it executes a **break** statement, or when the end of the **switch** statement is reached
 - When the computer executes the statements after a case label, it continues until a **break** statement is reached
 - If the **break** statement is omitted, then after executing the code for one case, the computer will go on to execute the code for the next case
 - If the **break** statement is omitted inadvertently, the compiler will not issue an error message



switch Statement Rules

The switch-expression must yield a value of char, byte, short, or int type and must always be enclosed in parentheses.

The value1, ..., and valueN must have the same data type as the value of the switch-expression. The resulting statements in the case statement are executed when the value in the case statement matches the value of the switch-expression. Note that value1, ..., and valueN are constant expressions, meaning that they cannot contain variables in the expression, such as $1 + x$.

```
switch (switch-expression) {  
    case value1:  
        statement(s)1;  
        break;  
    case value2:  
        statement(s)2;  
        break;  
    ...  
    case valueN:  
        statement(s)N;  
        break;  
    default:  
        statement(s)-for-default;  
}
```

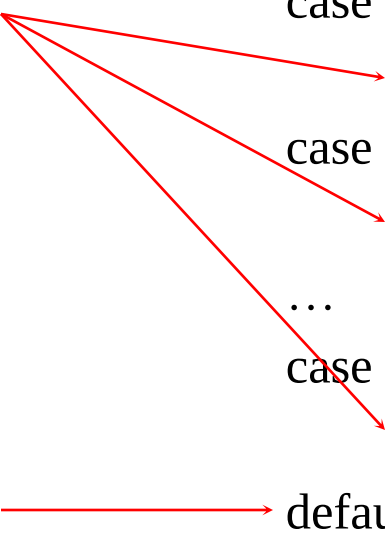


switch Statement Rules

The keyword break is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.

The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.

```
switch (switch-expression) {  
    case value1: statement(s)1;  
        break;  
    case value2: statement(s)2;  
        break;  
    ...  
    case valueN: statement(s)N;  
        break;  
    default: statement(s)-for-default;  
}
```



When the value in a **case** statement matches the value of the **switch-expression**, the statements *starting from this case* are executed until either a **break** statement or the end of the **switch** statement is reached.

The **switch** Statement

```
switch (Controlling_Expression)
{
    case Case_Label_1:
        Statement_Sequence_1
        break;
    case Case_Label_2:
        Statement_Sequence_2
        break;

    case Case_Label_n:
        Statement_Sequence_n
        break;
    default:
        Default_Statement_Sequence
        break;
}
```



Trace switch statement

Suppose day is 2:

```
switch (day) {  
    case 1:  
    case 3:  
        System.out.println("Weekday");  
        break;  
    case 2:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
    default:  
        ;  
}
```



Trace switch statement

Match case 2

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```



Trace switch statement

Fall through case 3

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```



Trace switch statement

Fall through case 4

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```



Trace switch statement

Fall through case 5

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```



Trace switch statement

Encounter break

```
switch (day) {  
  case 1:  
  case 2:  
  case 3:  
  case 4:  
  case 5: System.out.println("Weekday"); break;  
  case 0:  
  case 6: System.out.println("Weekend");  
}
```



Trace switch statement

Exit the statement

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```



The **switch** Statement Example

```
int month=-3;
switch(month) {
    case 1:
        System.out.println("January");
        break;
    case 2:
        System.out.println("February");
        break;
    □ case 3:
        System.out.println("March");
        break;
    case 4:
        System.out.println("April");
        break;
    case 5:
        System.out.println("May");
        break;
    case 6:
        System.out.println("June");
        break;
```



The **switch** Statement Example

case 7:

```
    System.out.println("July");  
    break;
```

case 8:

```
    System.out.println("August");  
    break;
```

case 9:

```
    System.out.println("September");  
    break;
```

case 10:

```
    System.out.println("October");  
    break;
```

case 11:

```
    System.out.println("November");  
    break;
```

case 12:

```
    System.out.println("December");  
    break;
```

default:

```
    System.out.println("Invalid Month");  
    break;
```

```
}
```



The **switch** Statement Example

```
char sClass='B';
switch (sClass) {
    case 'B': case 'b':
        System.out.printf("Battleship\n");
        break;
    case 'C': case 'c':
        System.out.printf("Cruiser\n");
        break;
    case 'D': case 'd':
        System.out.printf("Destroyer\n");
        break;
    case 'F': case 'f':
        System.out.printf("Frigate\n");
        break;
    default:
        System.out.printf("Unknown ship class %c\n", sClass);
        break;
}
```



Ternary operator

(testCondition) ? value1 : value2;

As described in the Sun documentation, this statement can be read as "If testCondition is true, assign the value of value1 to result; otherwise, assign the value of value2 to result."

```
if (x > 0)
    y = 1
else
    y = -1;
```

is equivalent to ➔ `y = (x > 0) ? 1 : -1;`



Operator Precedence

The expression in the parentheses (..) is evaluated first

`var++`, `var--`

`+`, `-` (Unary plus and minus), `++var`, `--var`

`(type)` Casting

`!` (Not)

`*`, `/`, `%` (Multiplication, division, and remainder)

`+`, `-` (Binary addition and subtraction)

`<`, `<=`, `>`, `>=` (Relational operators)

`==`, `!=`; (Equality)

`^` (Exclusive OR)

`&&` (Conditional AND) Short-circuit AND

`||` (Conditional OR) Short-circuit OR

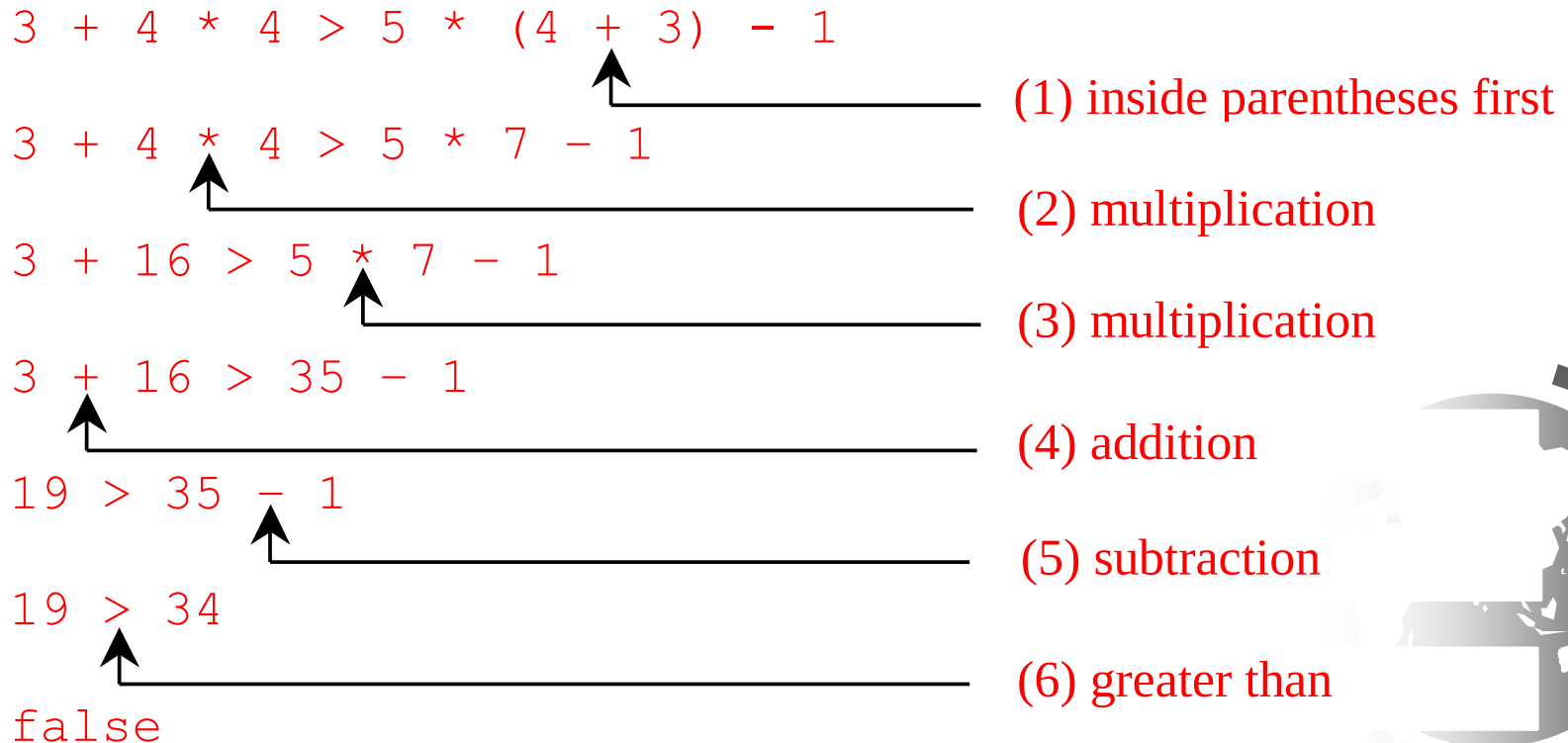
`=`, `+=`, `-=`, `*=`, `/=`, `%=` (Assignment operator)

When two operators with the same precedence → *left-associative*. However, assignment operators are right associative



Example

Applying the operator precedence and associativity rule, the expression $3 + 4 * 4 > 5 * (4 + 3) - 1$ is evaluated as follows:



Debugging

Logic errors are called *bugs*. The process of finding and correcting errors is called *debugging*.

For a large, complex program, the most effective approach for debugging is to use *a debugger utility*.

Debugger is a program that facilitates debugging. You can use a debugger to

- Execute a single statement at a time.
- Trace into or stepping over a method.
- Set breakpoints.
- Display variables.
- Display call stack.
- Modify variables.

